

Frozen Lake from First Principles Using Q-Learning

Technical Report — DSCD 614 Reinforcement Learning, Assignment 1

University of Ghana, Department of Computer Science

1. Introduction

Reinforcement Learning (RL) is a paradigm of machine learning in which an agent learns to act by interacting with an environment, receiving a scalar reward signal, and adjusting its behaviour to maximise expected cumulative reward. Unlike supervised learning, no labelled “correct action” is provided; the agent must discover good behaviour through trial and error.

This report documents a from-scratch implementation of Q-Learning applied to Frozen Lake, an 8×8 grid-world in which an agent must travel from a Start cell to a Goal cell while avoiding Holes. No reinforcement learning framework (Gymnasium, OpenAI Gym, Stable-Baselines, RLlib) was used. The environment, the agent, the training loop, and the evaluation harness were all implemented in plain Python and NumPy.

2. Environment Design

The environment is implemented as the class `FrozenLakeEnv`, exposing `reset()`, `step(action)`, `render()`, `get_state()`, and `is_terminal()`, as required.

2.1 State and Action Representation

Each of the 64 grid cells is mapped to a single integer state index via $\text{state} = \text{row} \times \text{n_cols} + \text{col}$, which keeps the Q-table a simple (64, 4) NumPy array. The four actions follow the assignment's specification exactly: 0 = Left, 1 = Down, 2 = Right, 3 = Up. Moves that would take the agent off the grid are clamped (the agent stays in place).

2.2 Reward Structure

The reward structure is sparse and standard for this benchmark: 0 for every non-terminal step, 0 for falling into a Hole (episode terminates), and +1 for reaching the Goal (episode terminates). This sparsity is what makes the credit-assignment problem non-trivial — the single terminal reward must be propagated backward through every state along the successful path.

3. Q-Learning Algorithm

Q-Learning is a model-free, off-policy temporal-difference algorithm that learns action-value estimates $Q(s,a)$: the expected discounted return of taking action a in state s and acting optimally thereafter. The agent's Q-table is initialised to all zeros and updated after every transition using exactly the rule specified in the assignment:

$$Q(s,a) \leftarrow Q(s,a) + \alpha [r + \gamma \max_{a'} Q(s',a') - Q(s,a)]$$

Here α (the learning rate) controls how strongly new information overrides prior estimates, γ (the discount factor) controls the weight given to future versus immediate reward, and the bracketed term is the TD-error — the gap between the agent's current estimate and a better-informed bootstrapped target. When the next state is terminal, the bootstrapped value $\max Q(s',a')$ is treated as zero.

3.1 Exploration Strategy

Action selection follows ϵ -greedy exploration: with probability ϵ a uniformly random action is chosen (explore); otherwise the highest-value action is chosen, with ties broken randomly (exploit). ϵ begins at 1.0 and decays multiplicatively after every episode toward a floor of 0.01, shifting the agent from exploration to exploitation as training progresses.

4. Training Methodology

The agent was trained for 20,000 episodes per configuration (40,000 for the stochastic bonus run) with a per-episode cap of 200 steps. As required, three hyperparameters were varied independently to study their effect on learning:

Hyperparameter	Values tested	Default
Learning rate (α)	0.02, 0.1, 0.5	0.1
Discount factor (γ)	0.80, 0.99, 0.999	0.99
Epsilon decay rate	0.999, 0.9995, 0.9999	0.9995

Episode rewards, success/failure outcomes, and the epsilon value were logged at every episode to enable the analysis and plots below.

5. Experimental Results

5.1 Baseline Convergence

With the default hyperparameters ($\alpha=0.1$, $\gamma=0.99$), the rolling 100-episode success rate climbs from 0% to a stable 100% within roughly 5,000–9,000 episodes, closely tracking the decay of ϵ from 1.0 to its floor of 0.01.

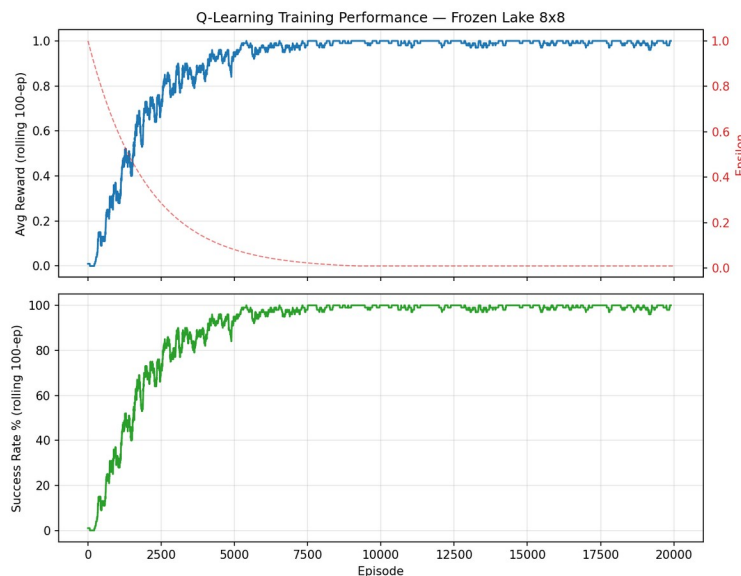


Figure 1. Rolling average reward (top, with epsilon overlay) and success rate (bottom) over training.

5.2 Effect of Learning Rate and Discount Factor

All tested combinations of α and γ eventually reach 100% success on this deterministic map — the state space is small enough that exhaustive exploration guarantees convergence regardless of these settings. The settings primarily affect convergence speed: episodes required to sustain a 95% rolling success rate were 3,721 ($\alpha=0.02$), 4,328 ($\alpha=0.1$, default), 4,843 ($\alpha=0.5$), 3,924 ($\gamma=0.80$), and 4,537 ($\gamma=0.999$). A very high learning rate (0.5) converged slightly slower and noisier than moderate values, consistent with the known instability risk of large step sizes in TD learning.

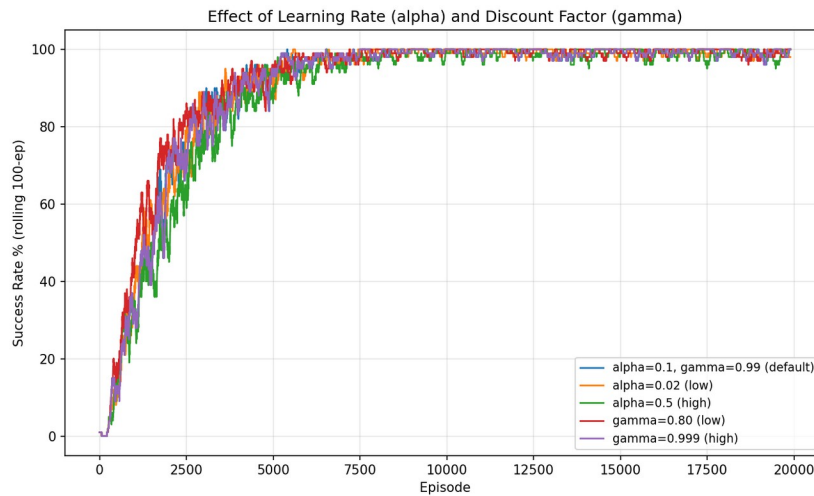


Figure 2. Rolling success rate for different α and γ settings.

5.3 Bonus — Exploration Strategy Comparison (Option C)

Pure ϵ -greedy (fixed $\epsilon=0.1$, no decay) was compared against decaying ϵ -greedy. Pure ϵ -greedy rises almost immediately because it spends far less time near $\epsilon=1.0$, but it plateaus around 90–95% and never stabilises, since 10% of actions remain random forever — occasionally stepping into a hole even with an otherwise-optimal Q-table. Decaying ϵ -greedy converges more slowly but reaches a clean, stable 100%, since exploration is eventually switched off almost entirely.

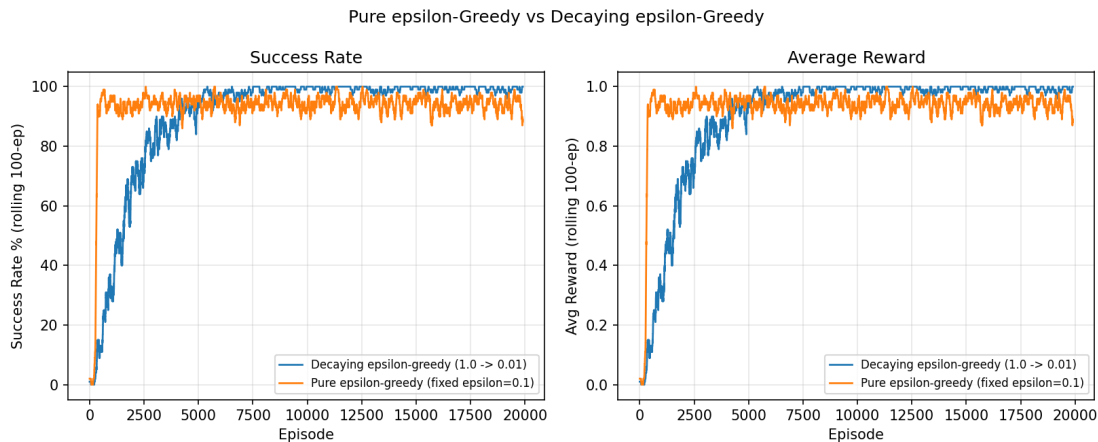


Figure 3. Pure ϵ -greedy vs. decaying ϵ -greedy: success rate and average reward.

5.4 Bonus — Stochastic Transitions (Option A)

A slippery variant was implemented in which an executed action has a 20% chance of slipping into a perpendicular direction. Training directly on this stochastic environment for 40,000 episodes converges more slowly and noisily than the deterministic case, plateauing near 90% rather than 100%, since occasional unlucky slips into a hole are unavoidable even under an optimal policy. Critically, when the agent trained on the deterministic map was evaluated under the same stochastic dynamics without retraining, its success rate dropped to 67.4% (337/500), whereas the agent trained directly under stochastic dynamics achieved 100% (500/500) success when evaluated under those same dynamics — it learned a more cautious route that avoids narrow corridors next to holes, demonstrating that the agent adapts its policy to the dynamics it is trained under.

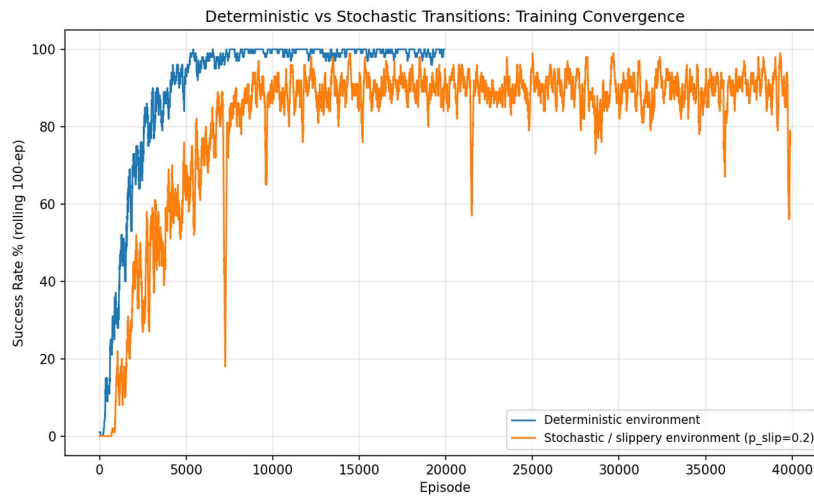


Figure 4. Training convergence: deterministic vs. stochastic ($p_{\text{slip}}=0.2$) transitions.

6. Learned Policy

The greedy policy extracted from the final Q-table of the default run, displayed in grid form (H = hole, G = goal, arrows = greedy action):

```

↓ ↓ ↓ ↓ ↓ ↓ ↓ ↓
→ → → → → ↓ ↓ ↓
↑ ↑ ↑ H → → → ↓
↑ ↑ ↑ H ↑ → → ↓
↑ ↑ ↑ H → → → ↓
↑ H H → ↑ ↑ H ↓
← H ← ← H ↑ H ↓
← ← ← H ← ← ← G

```

The agent learned to route down the left and top edges, then cut diagonally across open corridors, finally approaching the goal from the left along the bottom row — a 14-step shortest safe path that avoids every hole.

Metric (200 greedy episodes)	Value
Success Rate	100.00%
Average Reward	1.0000
Number of Successful Runs	200
Number of Failures	0
Average Episode Length	14.00 steps

7. Challenges Encountered

- Tie-breaking in greedy action selection: initialising the Q-table to zeros means many actions are tied early in training. Always choosing the first maximal action (e.g. via `argmax`) biases the agent toward action 0; this was resolved by sampling uniformly among all tied best actions.
- Distinguishing convergence from final accuracy: because the deterministic map is small, nearly every hyperparameter setting eventually reaches 100% success, masking real differences. Episodes-to-95%-success was used as a more sensitive comparison metric than final success rate alone.

- Reward sparsity: with reward 0 on every step but the goal, early training produces almost no learning signal, since random exploration rarely reaches the goal by chance in a large grid. This was mitigated purely by patience (sufficient episodes) and a high initial epsilon, without reward shaping, to stay faithful to the assignment's first-principles requirement.
- Stochastic transitions slow convergence substantially: the slippery variant required roughly double the episodes (40,000 vs 20,000) to reach a stable plateau, since the same action can lead to different outcomes, requiring more samples to estimate Q-values reliably.

8. Conclusion

This project implemented Frozen Lake and a tabular Q-Learning agent entirely from first principles, without relying on any reinforcement learning framework. The agent reliably learns an optimal, hole-avoiding policy, reaching a 100% success rate under default hyperparameters and evaluation. Experiments varying the learning rate, discount factor, and exploration schedule confirmed that, on this small deterministic map, final performance is robust to these choices, while convergence speed is more sensitive to them. The bonus comparisons further showed that decaying ϵ -greedy outperforms a fixed exploration rate at convergence quality, and that training directly under stochastic dynamics produces a measurably more robust policy than training deterministically and hoping it generalises.